Lab 1: Model Selection and Comparative Analysis

Manual Grid Search vs. Scikit-learn GridSearchCV for Classifier Hyperparameter Tuning

Name: Mohamed Riaz
Registration Number: PES1PGE25DS037
Department: Data Science & AI
Course: UX25CS635A – Supervised Machine Learning: Classification
Submission Date: 14 August 2026

1. Introduction

This lab implements a complete machine learning pipeline for binary classification and compares two approaches to hyperparameter tuning: a Grid Search implemented from scratch using nested loops and stratified k-fold cross-validation, and scikit-learn's built-in **GridSearchCV**. Three classifiers – Decision Tree, k-Nearest Neighbours (kNN), and Logistic Regression – are tuned inside a **StandardScaler** → **SelectKBest** → **Classifier** pipeline and evaluated on two datasets: **HR Attrition** and **Banknote Authentication**. Individual models are also combined into soft-voting ensembles, and all models are compared using Accuracy, Precision, Recall, F1-Score and ROC AUC.

2. Dataset Description

2.1 HR Attrition

Predicts whether an employee will leave the company (target: **Attrition**, Yes/No). The raw IBM dataset has 1470 instances and 34 work-related and personal attributes (e.g. age, monthly income, job role, overtime, years at company). After dropping the constant/ID column *EmployeeNumber* and one-hot encoding categorical variables, the feature space expands to **46 numeric features**. The target is imbalanced, with an attrition rate of about 16.1% in the training split (1029 train / 441 test, 70/30 stratified split).

2.2 Banknote Authentication

Distinguishes genuine from forged banknotes (target: **class**, 0 = authentic, 1 = forged) using **4 features** extracted from wavelet-transformed banknote images: variance, skewness, curtosis and entropy. The dataset has 1372 instances (960 train / 412 test, 70/30 stratified split) and is close to balanced, with a forged rate of about 44.5% in the training split.

3. Methodology

Hyperparameter Tuning & Grid Search: Hyperparameters (e.g. tree depth, number of neighbours, regularization strength) are not learned from data directly and must be chosen externally. Grid Search evaluates every combination in a predefined parameter grid and picks the combination that yields the best cross-validated score.

K-Fold Cross-Validation: To get a robust performance estimate for each hyperparameter combination, the training data is split into 5 stratified folds. Each fold is used once as a validation set while the pipeline is fit on the remaining 4 folds; the mean ROC AUC across the 5 folds is used as the score for that combination.

Pipeline: Every model uses the same three-stage pipeline: *StandardScaler* (zero mean, unit variance) → *SelectKBest* (top-k features by the f_classif ANOVA F-statistic, with k itself tuned) → *Classifier* (Decision Tree / kNN / Logistic Regression). Fitting the scaler and feature selector only within each training fold (never on the validation fold) prevents data leakage.

Part 1 – Manual Implementation: For each classifier, all combinations of its parameter grid are generated with *itertools.product*. For every combination, the pipeline is rebuilt, its parameters set with *set_params*, fit on each training fold and scored with *roc_auc_score* on the corresponding validation fold. The combination with the highest mean AUC is kept and a final pipeline is refit on the full training set.

Part 2 – Scikit-learn Implementation: The same pipeline and parameter grid are passed to *GridSearchCV* with *scoring='roc_auc'* and the same *StratifiedKFold(n_splits=5)* cross-validator, then fit on the training data. GridSearchCV parallelizes the search with *n_jobs=-1* and directly exposes *best_estimator_*, *best_params_* and *best_score_*.

Voting Classifiers: The three tuned models are combined into a soft-voting ensemble. In the manual implementation this is done by hand (majority vote of predicted class labels, average of predicted probabilities); in the built-in implementation, scikit-learn's *VotingClassifier(voting='soft')* is used, which averages predicted probabilities and assigns the class directly — a subtle behavioural difference discussed in Section 4.

4. Results and Analysis

4.1 HR Attrition

Best hyperparameters found (identical for manual and built-in search, as expected since both use the same StratifiedKFold(n_splits=5, random_state=42) splitter and the same grid): Decision Tree → k=5, max_depth=3, min_samples_split=2 (CV AUC 0.7152); kNN → k=10, n_neighbors=9, weights=distance (CV AUC 0.7226); Logistic Regression → k=all, C=0.1, penalty=l2 (CV AUC 0.8329).

Model	Method	Accuracy	Precision	Recall	F1-Score	ROC AUC
Decision Tree	Manual	0.8231	0.3333	0.0986	0.1522	0.7107
Decision Tree	Built-in	0.8231	0.3333	0.0986	0.1522	0.7107
kNN	Manual	0.8186	0.3784	0.1972	0.2593	0.7236
kNN	Built-in	0.8186	0.3784	0.1972	0.2593	0.7236
Logistic Regression	Manual	0.8798	0.7368	0.3944	0.5138	0.8187
Logistic Regression	Built-in	0.8798	0.7368	0.3944	0.5138	0.8187
Voting Classifier	Manual	0.8299	0.4231	0.1549	0.2268	0.7966
Voting Classifier	Built-in	0.8458	0.5556	0.2113	0.3061	0.7966

Manual vs. built-in: for all three individually-tuned classifiers the two implementations are identical on every metric, confirming the manual search reproduces GridSearchCV exactly when the same grid and CV splitter are used. The *Voting Classifier* is the only place they diverge (Accuracy 0.8299 vs 0.8458): both average the same predicted probabilities (hence identical ROC AUC = 0.7966), but the manual implementation assigns the final label from a majority vote of each model's hard predictions, while scikit-learn's VotingClassifier(voting='soft') thresholds the averaged probability directly at 0.5 – a small but real algorithmic difference, not a bug.

Best model: Logistic Regression achieves the highest test ROC AUC (0.8187), outperforming both the Decision Tree, kNN and even the Voting Classifier. HR Attrition is a mostly linear-separable, highly imbalanced problem (~16% positive class) with many one-hot encoded categorical features; Logistic Regression's L2-regularized linear decision boundary generalizes better here than the higher-variance Decision Tree/kNN, and dragging those weaker models into the vote actually pulls the ensemble's performance down.

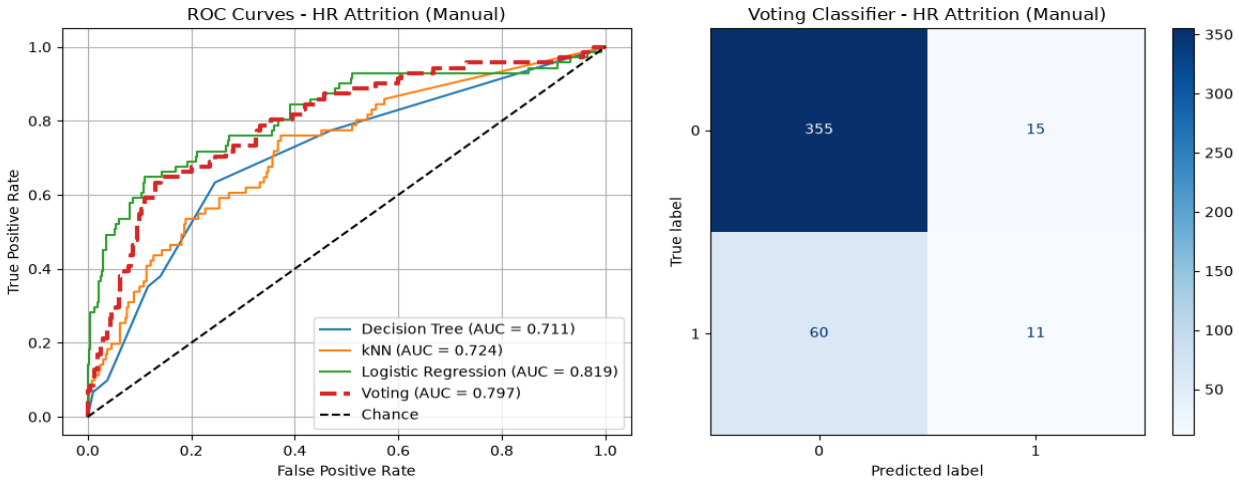

Figure 1: ROC curves and confusion matrix – HR Attrition (Manual).

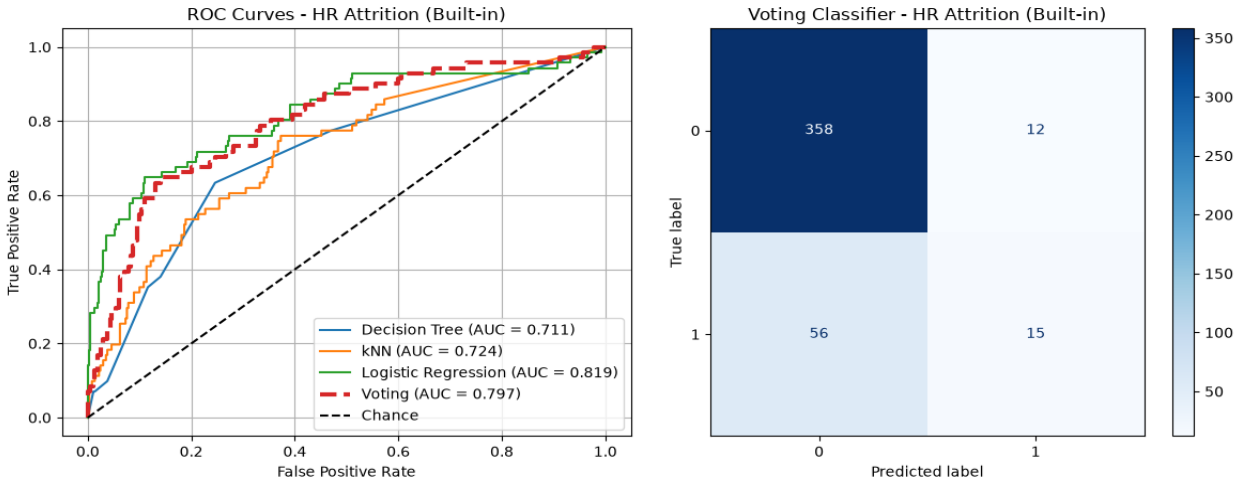

Figure 2: ROC curves and confusion matrix – HR Attrition (Built-in).

4.2 Banknote Authentication

Best hyperparameters found (again identical across manual and built-in search): Decision Tree → k=3, max_depth=10, min_samples_split=10 (CV AUC 0.9869); kNN → k=3, n_neighbors=5, weights=distance (CV AUC 1.0000); Logistic Regression → k=3, C=10, penalty=l2 (CV AUC 0.9995).

Model	Method	Accuracy	Precision	Recall	F1-Score	ROC AUC
Decision Tree	Manual	0.9854	0.9784	0.9891	0.9837	0.9856
Decision Tree	Built-in	0.9854	0.9784	0.9891	0.9837	0.9856
kNN	Manual	1.0000	1.0000	1.0000	1.0000	1.0000
kNN	Built-in	1.0000	1.0000	1.0000	1.0000	1.0000
Logistic Regression	Manual	0.9903	0.9786	1.0000	0.9892	0.9999
Logistic Regression	Built-in	0.9903	0.9786	1.0000	0.9892	0.9999
Voting Classifier	Manual	1.0000	1.0000	1.0000	1.0000	1.0000
Voting Classifier	Built-in	1.0000	1.0000	1.0000	1.0000	1.0000

Manual vs. built-in: every model, including the Voting Classifier this time, matches exactly between the two implementations. With only 4 highly-informative wavelet features and a near-perfectly separable dataset, the averaged-probability and majority-vote strategies agree on every test sample, so the earlier source of divergence does not appear here.

Best model: kNN and the Voting Classifier both reach a perfect test ROC AUC of 1.0000; Logistic Regression is close behind (0.9999) and the Decision Tree is the weakest (0.9856). The banknote features are strongly and fairly simply correlated with authenticity, so a distance-based method like kNN with only 3 selected features can separate the classes almost perfectly, and the Decision Tree's axis-aligned splits are the least efficient way to capture that structure, explaining why it trails the other two.

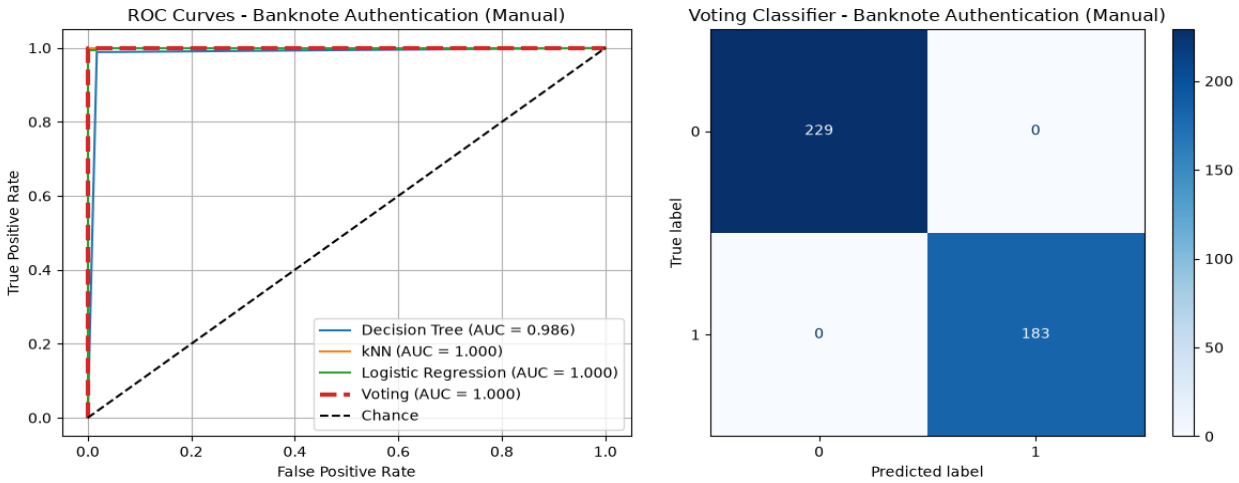

Figure 3: ROC curves and confusion matrix – Banknote Authentication (Manual).

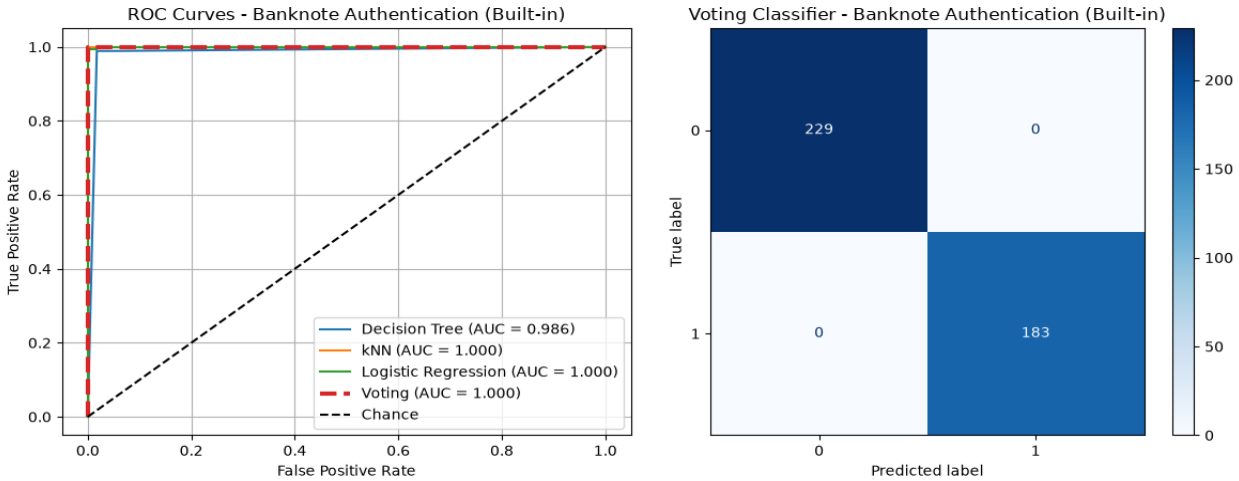

Figure 4: ROC curves and confusion matrix – Banknote Authentication (Built-in).

5. Screenshots of Execution Output

Full, cell-by-cell console output (grid search progress, best parameters, per-model and voting classifier metrics) is preserved in the executed notebooks under *solution/notebooks/*. Representative excerpts are reproduced below.

HR Attrition (excerpt)

```
RUNNING MANUAL GRID SEARCH FOR HR ATTRITION
--- Manual Grid Search for Decision Tree ---
Best parameters for Decision Tree: {'feature_selection__k': 5, 'classifier__max_depth': 3,
'classifier__min_samples_split': 2}
Best cross-validation AUC: 0.7152
--- Manual Grid Search for Logistic Regression ---
Best parameters for Logistic Regression: {'feature_selection__k': 'all', 'classifier__C': 0.1,
'classifier__penalty': 'l2'}
Best cross-validation AUC: 0.8329

EVALUATING BUILT-IN MODELS FOR HR ATTRITION
Logistic Regression:
Accuracy: 0.8798 Precision: 0.7368 Recall: 0.3944 F1-Score: 0.5138 ROC AUC: 0.8187
Built-in Voting Classifier:
Accuracy: 0.8458, Precision: 0.5556, Recall: 0.2113, F1: 0.3061, AUC: 0.7966
```

Banknote Authentication (excerpt)

```
RUNNING BUILT-IN GRID SEARCH FOR BANKNOTE AUTHENTICATION
Best params for kNN: {'classifier__n_neighbors': 5, 'classifier__weights': 'distance',
'feature_selection__k': 3}
Best CV score: 1.0000

EVALUATING BUILT-IN MODELS FOR BANKNOTE AUTHENTICATION
kNN:
Accuracy: 1.0000 Precision: 1.0000 Recall: 1.0000 F1-Score: 1.0000 ROC AUC: 1.0000
Built-in Voting Classifier:
Accuracy: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000, AUC: 1.0000
```

6. Conclusion

- The manual grid search reproduces scikit-learn's GridSearchCV exactly for every individually-tuned classifier on both datasets, confirming a correct understanding of nested cross-validation and pipeline parameter passing.
- GridSearchCV is far more practical for real work: it is shorter to write, parallelizes the search across folds/combinations with n_jobs=-1, and exposes best_estimator_/best_params_/best_score_ directly, whereas the manual version required explicitly looping over folds and combinations and re-fitting the final pipeline by hand.
- Voting classifiers help most when the base models are already strong and diverse; on HR Attrition, voting actually underperformed the single best model (Logistic Regression) because it was dragged down by the much weaker Decision Tree and kNN models, while on the near-perfectly-separable Banknote dataset voting matched the best individual model.
- The only observed manual-vs-built-in discrepancy was in the Voting Classifier on HR Attrition, and it was traced to a genuine algorithmic difference (majority-vote-of-labels vs. threshold-on-averaged-probability) rather than an implementation bug – a useful reminder that scikit-learn's convenience functions can encode subtly different design choices than a naive from-scratch implementation.

- Overall, this lab reinforced how feature scaling, feature selection and hyperparameter tuning must be chained inside a single cross-validated pipeline to avoid data leakage, and how the right classifier choice depends heavily on the dataset's separability and class balance.